

Banker's Algorithm for Deadlock Avoidance Simulator

CS2006 – Operating Systems
Spring 2026

National University of Computer and Emerging Sciences (FAST-NUCES)

Project Team:

Murtaza Hunaid Terai | 24K-0782

Adnan Hatim | 24K-0656

Supervised By:

Miss Rabia Ahmed Ansari

Table of Contents

1. Introduction
2. Literature Review
3. System Design / Architecture
4. Implementation
5. Testing and Results
6. Discussion
7. Conclusion
8. References

1. Introduction

1.1 Background

In modern operating systems, multiple processes compete for a finite set of resources. When processes hold resources while waiting for additional ones held by other processes, the system may enter a deadlock—a state where no process can proceed. Deadlock is one of the most critical issues in resource management, potentially causing entire subsystems to freeze.

Deadlock handling strategies generally fall into four categories: prevention, avoidance, detection and recovery, and ignorance (the Ostrich algorithm). Among these, deadlock avoidance offers a pragmatic balance between system utilization and safety by allowing resource requests dynamically while ensuring the system never enters an unsafe state.

1.2 The Banker's Algorithm

The Banker's Algorithm, originally proposed by Edsger W. Dijkstra in 1965, is a classical deadlock avoidance algorithm. It models the operating system as a banker who must ensure that loans (resource allocations) never leave the bank in a state where it cannot satisfy the maximum potential needs of all clients (processes). The algorithm requires each process to declare its maximum resource demand in advance. Before granting any request, the system simulates the allocation and checks whether a safe sequence exists—an order in which all processes can complete without deadlock.

1.3 Project Objectives

This project implements a fully functional Banker's Algorithm Simulator that:

- Models a multi-process, multi-resource environment.
- Dynamically constructs and displays Allocation, Maximum, and Need matrices.
- Executes the Safety Algorithm step-by-step to determine safe sequences.
- Processes runtime resource requests with proactive deadlock avoidance verification.
- Automatically rolls back hypothetical allocations that would lead to unsafe states.
- Provides terminal-based visualization, interactive input, persistent logging, and performance analysis.

1.4 Success Criteria

The simulator is considered successful if it:

- Correctly identifies safe and unsafe states.
- Grants only those requests that preserve system safety.
- Denies unsafe requests with clear reasoning and rollback.
- Provides real-time visual feedback and persistent logs.
- Operates interactively with configurable parameters.

2. Literature Review

2.1 Foundational Work

The theoretical foundation of deadlock avoidance was established by Edsger W. Dijkstra in his 1965 paper, "Solution of a problem in concurrent programming control" (Communications of the ACM). Dijkstra introduced the Banker's Algorithm as an elegant solution to the resource allocation problem, framing it as a credit-allocation model where the operating system must never commit resources that could lead to an irrecoverable state.

2.2 Modern Textbook Treatment

Silberschatz, Galvin, and Gagne (2018), in *Operating System Concepts* (10th Edition), provide the canonical modern treatment of the Banker's Algorithm. They formalize the data structures—Allocation, Maximum, Need, and Available—and present the Safety Algorithm as a two-step procedure: (1) find a process whose needs can be satisfied by currently available resources, and (2) simulate its completion, reclaim its resources, and repeat. The textbook also distinguishes between the Safety Algorithm (system-wide check) and the Resource-Request Algorithm (per-request check).

Tanenbaum and Bos (2014), in *Modern Operating Systems* (4th Edition), extend this discussion by contrasting avoidance with detection and prevention strategies. They note that while the Banker's Algorithm is theoretically optimal for avoidance, its practical limitation lies in the requirement that processes must declare maximum needs in advance—a condition not always feasible in general-purpose systems.

2.3 Educational Implementations

Educational resources such as GeeksforGeeks (2023) have contributed significantly to the pedagogical dissemination of the algorithm by providing worked examples, trace tables, and code implementations in multiple languages. These resources confirm that the algorithm remains a staple of undergraduate operating systems curricula worldwide due to its clarity and illustrative power.

2.4 Synthesis

The literature consistently identifies the Banker's Algorithm as a cornerstone of deadlock avoidance theory. While modern operating systems often employ simpler strategies (such as resource ordering or deadlock detection), the algorithm remains invaluable for understanding the fundamental trade-offs between resource utilization and system safety. This project contributes to this body of work by providing a complete, modular, interactive simulator that bridges theoretical understanding with hands-on experimentation.

3. System Design / Architecture

3.1 Architectural Overview

The simulator is designed as a modular, multi-file C application with clear separation of concerns. The architecture follows a layered approach:

Presentation Layer (Visualization) ← Terminal UI, matrix display
Interaction Layer (Interactive Input) ← Menu system, user prompts
Business Logic Layer ← Safety algorithm, request handler
Data Layer (Data Structures) ← SystemState, matrices, logging
Persistence Layer (Logger) ← File I/O, timestamped records

3.2 Core Data Structures

The central data structure, SystemState, encapsulates the entire system configuration:

- total_resources[]: Total instances of each resource type.
- available[]: Currently unallocated instances.
- allocation[][]: Resources currently held by each process.
- maximum[][]: Peak demand declared by each process.
- need[][]: Remaining resources each process may request (Need = Maximum - Allocation).
- process_active[]: Boolean flags indicating active processes.

A SafetyResult structure captures the output of the Safety Algorithm, including the safe sequence and a safety flag. A PerformanceMetrics structure tracks runtime statistics.

3.3 Module Decomposition

The system is decomposed into eleven source files organized by functionality:

Module	Responsibility
main.c	Entry point, mode selection (Interactive / Demo / Custom Demo)
data_structures.c	System initialization, process lifecycle, Need matrix computation
safety_algorithm.c	Core Safety Algorithm and hypothetical state checking
request_handler.c	Request validation, hypothetical allocation, safety verification
rollback.c	State snapshot capture and restoration
performance.c	Metrics collection and report generation
resource_manager.c	Resource type definition and availability tracking
visualization.c	Formatted terminal output for matrices and safe sequences
interactive_input.c	Menu-driven interactive command interface
logger.c	Persistent timestamped logging to

	banker_log.txt
banker.h	Shared header with declarations, constants, and structures

3.4 Algorithm Flow

The resource request workflow follows the classical Banker's Algorithm:

- Validation: Check if $\text{Request} \leq \text{Need}[\text{pid}]$ and $\text{Request} \leq \text{Available}$.
- Hypothetical Allocation: Tentatively subtract from Available, add to Allocation[pid], subtract from Need[pid].
- Safety Check: Execute the Safety Algorithm on the hypothetical state.
- Decision: If safe, commit the allocation. If unsafe, restore the snapshot and deny the request.

3.5 Design Decisions

- Language: C99 was chosen for direct memory control, portability, and alignment with systems programming conventions.
- Modularity: Each file compiles independently, enabling parallel development and unit testing.
- Maximum Limits: $\text{MAX_PROCESSES} = 20$ and $\text{MAX_RESOURCES} = 10$ provide sufficient flexibility for educational scenarios while keeping memory overhead negligible.
- Logging: All state transitions are written to a persistent file for post-hoc analysis and debugging.

4. Implementation

4.1 Development Environment

- Language: C (C99 standard)
- Compiler: GCC with flags -Wall -Wextra -std=c99 -g
- Build System: GNU Make
- Platform: Linux / Unix-compatible systems

4.2 Key Implementation Details

4.2.1 Safety Algorithm (safety_algorithm.c)

The Safety Algorithm is implemented as a greedy search:

```
Work = Available; Finish[i] = false for all active processes; while (not all finished) { find process i where Need[i] ≤ Work and Finish[i] == false; if found: Work = Work + Allocation[i]; Finish[i] = true; add i to safe_sequence; else: return UNSAFE; } return SAFE;
```

A secondary function, `check_safety_with_temp()`, operates on temporary state copies to evaluate hypothetical allocations without mutating the live system state.

4.2.2 Request Handling (request_handler.c)

The `request_resources()` function orchestrates the complete request lifecycle:

- Validates bounds against Need and Available.
- Invokes `save_state_snapshot()` to preserve the current state.
- Calls `perform_hypothetical_allocation()` to modify temporary copies.
- Runs `check_safety_with_temp()` and measures execution time.
- Commits or rolls back based on the safety result.

4.2.3 Rollback Mechanism (rollback.c)

Rollback is achieved through deep-copy snapshots. Two functions manage this:

`save_state_snapshot()` copies the current state into temporary arrays, and `restore_state_from_snapshot()` copies them back. If a hypothetical allocation is rejected, the system state remains entirely unaffected because all modifications were applied only to temporary arrays.

4.2.4 Visualization (visualization.c)

The terminal UI renders matrix tables with aligned columns for Allocation, Maximum, and Need; vector displays for Available and Total resources; safe sequence boxes using ASCII art for visual clarity; and process status tables showing active/inactive states and current holdings.

4.2.5 Logging (logger.c)

Every significant event is logged with a timestamp, including process addition/removal, resource requests (granted or denied, with reasoning), safe and unsafe state detections, resource releases, and system resets. Log entries follow a structured format with timestamps, process names, and resource vectors.

4.2.6 Performance Metrics (performance.c)

The system collects total requests processed, granted, and denied; grant and denial rates; safety check execution times (average, minimum, maximum); and total time spent in safety checks. Reports can be displayed in the terminal or exported to a text file.

4.3 Build Configuration

The Makefile supports multiple targets: make all for standard build, make rebuild for clean and rebuild, make run for build and execute, make debug for build with debug symbols, and make release for optimized build.

5. Testing and Results

5.1 Test Strategy

Testing was conducted at two levels: Unit Testing, where individual modules (e.g., safety check, snapshot/restore) were tested in isolation; and Integration Testing, where full scenarios were executed via the Demo and Custom Demo modes to verify end-to-end correctness.

5.2 Demo Scenario: Classic Textbook Example

A 5-process, 3-resource scenario was configured to match the canonical example from Operating System Concepts (Silberschatz et al., 2018). The following table summarizes the configuration:

Process	Maximum (R0,R1,R2)	Initial Allocation
P0	(7, 5, 3)	(0, 1, 0)
P1	(3, 2, 2)	(2, 0, 0)
P2	(9, 0, 2)	(3, 0, 2)
P3	(2, 2, 2)	(2, 1, 1)
P4	(4, 3, 3)	(0, 0, 2)

Total Resources: (10, 5, 7). Available after initialization: (3, 3, 2).

Result 1: Safety Check

The Safety Algorithm produced the safe sequence: $\langle P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2 \rangle$. This confirms the system was in a safe state after initialization.

Result 2: Valid Request

Process P1 requested (1, 0, 2). The simulator verified the request was within P1's Need and Available resources, hypothetically allocated the resources, ran the Safety Algorithm on the new state, found a valid safe sequence, and granted the request.

Result 3: Invalid Request (Unsafe)

Process P4 requested (3, 3, 0). The simulator detected that resource type R0 had only 2 available instances and denied the request immediately due to insufficient availability.

5.3 Custom Demo: Unsafe State Demonstration

A 3-process scenario was designed to force an unsafe state. After initial allocations, Process P0 requested (7, 4, 3). Although this amount was within P0's declared Need, the simulator determined that granting it would leave the system in an unsafe state where no process could complete. The simulator performed a hypothetical allocation, ran the Safety Algorithm, detected no safe sequence existed, rolled back the hypothetical allocation, and denied the request with clear reasoning. This successfully demonstrated the deadlock avoidance property.

5.4 Performance Results

The following table summarizes the performance metrics collected during testing:

Metric	Value
Total Requests Processed	7
Requests Granted	6
Requests Denied	1
Grant Rate	85.71%
Total Safety Checks	6
Average Safety Check Time	~0.001 ms
Maximum Safety Check Time	~0.001 ms

The algorithm executes in sub-millisecond time for the tested configurations, confirming its computational feasibility for small-to-medium-scale systems.

5.5 Log Verification

The generated `banker_log.txt` was manually inspected and confirmed to contain accurate timestamps for all events, complete safe sequence recordings, denial reasons for rejected requests, and process lifecycle events.

6. Discussion

6.1 Strengths

The simulator successfully demonstrates all mandatory and additional features specified in the project proposal:

- **Correctness:** The Safety Algorithm and request handler correctly identify safe and unsafe states, matching theoretical predictions.
- **Modularity:** The file-per-module architecture made parallel development efficient and the codebase maintainable.
- **Usability:** Three distinct modes (Interactive, Demo, Custom Demo) accommodate different use cases, from hands-on experimentation to automated verification.
- **Observability:** Terminal visualization and persistent logging provide excellent transparency into the algorithm's internal decisions.
- **Rollback Simulation:** The automatic rollback with reasoning display is a pedagogically valuable feature that makes the avoidance mechanism tangible.

6.2 Challenges Encountered

State Management: Ensuring that hypothetical allocations did not corrupt the live system state required careful design of the snapshot mechanism. Early attempts risked partial updates; this was resolved by performing all hypothetical changes on deep-copied temporary arrays.

Type Safety: During compilation, a type mismatch between `bool*` and `int*` in the `check_safety_with_temp()` function was detected by GCC's `-Wall -Wextra` flags. This was corrected by ensuring consistent use of `bool` arrays throughout the codebase.

Visualization Alignment: Formatting matrix columns for variable-digit numbers required dynamic padding calculations to maintain readability across different resource counts.

6.3 Limitations

- The simulator assumes that processes declare their maximum resource needs before execution, which may not be realistic for all real-world applications.
- It does not implement deadlock detection or recovery mechanisms; it focuses exclusively on avoidance.
- The terminal-based interface, while functional, lacks graphical widgets that might enhance accessibility for some users.

6.4 Learning Outcomes

This project reinforced several key operating systems concepts: the distinction between safe and unsafe states and why the latter does not necessarily imply deadlock, but must still be avoided; the computational cost of safety checking and the trade-offs involved in dynamic resource allocation; and the importance of clean modular design in systems programming.

7. Conclusion

This project presented the design, implementation, and testing of a Banker's Algorithm for Deadlock Avoidance Simulator. The system accurately models a multi-process, multi-resource environment and enforces deadlock avoidance through proactive safety verification. All core components—data structure management, the Safety Algorithm, request handling with rollback, terminal visualization, interactive input, logging, and performance analysis—were successfully implemented and validated.

Testing confirmed that the simulator correctly grants safe requests, denies unsafe requests with automatic rollback, and produces valid safe sequences consistent with established textbook examples. The sub-millisecond safety check times demonstrate that the algorithm remains computationally tractable for educational and small-scale systems.

Future Work

- Graphical User Interface (GUI): A web-based or desktop GUI using libraries such as GTK or a web frontend to render resource allocation graphs and safe sequence animations.
- Deadlock Detection Module: Extending the simulator to include a separate deadlock detection and recovery mode for comparative study.
- Randomized Stress Testing: Automated generation of random process-resource configurations to statistically verify correctness across a broader input space.
- Multi-threaded Simulation: Simulating concurrent process execution where requests arrive asynchronously, requiring synchronization of the safety check.

References

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- [2] Dijkstra, E. W. (1965). Solution of a problem in concurrent programming control. Communications of the ACM, 8(9), 569.
- [3] Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson.
- [4] GeeksforGeeks. (2023). Banker's Algorithm in Operating System. Retrieved from <https://www.geeksforgeeks.org/bankers-algorithm-in-operating-system/>
- [5] Coffman, E. G., Elphick, M., & Shoshani, A. (1971). System deadlocks. Computing Surveys, 3(2), 67–78.
- [6] Holt, R. C. (1972). Some deadlock properties of computer systems. ACM Computing Surveys, 4(3), 179–196.